

操作系统基础与程序运行环境

从裸硬件到受保护、可共享、可管理的执行环境

408 操作系统专题手册 OS-00

母问题：操作系统为什么存在？

一段普通程序为什么可以像“独占一台机器”一样运行，
而真实硬件明明有限、共享、异步，并且可能发生故障？

本册不从“操作系统有哪些功能”开始背诵，而从裸硬件不能直接满足程序需求开始推导：

$\text{Finite Physical Resources} \xrightarrow[\text{Protection + Policy}]{\text{Abstraction + Virtualization}} \text{Program-visible Managed Environment}$

全书继续沿用 OS 统一推演引擎：

$\text{State} \rightarrow \text{Transition} \rightarrow \text{Failure} \rightarrow \text{Property} \rightarrow \text{Mechanism} \rightarrow \text{Tradeoff}$
--

但 OS-00 的任务不是深入某个局部机制，而是建立后续所有 Topic 共用的坐标系、运行入口和边界。

术语预备：本册先固定这些词的含义

- **操作系统 (OS)**: 运行在硬件之上、为程序分配资源并提供受保护接口的软件系统；它不是某个单独的应用程序。
- **抽象 (Abstraction)**: 隐藏硬件实现细节，只向程序暴露稳定的对象和操作，例如把磁盘块组织成可按名字打开的文件。
- **虚拟化 (Virtualization)**: 用软件和硬件把有限物理资源表示成多个彼此隔离的逻辑资源，例如每个进程看到自己的虚拟地址空间。
- **保护/隔离 (Protection/Isolation)**: 规定一个执行主体能访问哪些对象，并阻止它越过权限边界修改其他主体的状态。
- **资源管理 (Resource Management)**: 在资源有限或多人竞争时，负责分配、回收、复用和调度 CPU、内存、设备及存储。
- **用户态/内核态 (User/Kernel Mode)**: CPU 的两种权限状态。用户态不能执行特权操作；内核态可以管理页表、设备和调度状态。
- **系统调用 (System Call)**: 用户程序请求 OS 服务的受控入口，例如 `read`、`open` 和 `fork`；它不是普通函数调用。
- **中断 (Interrupt)**: 由当前指令流之外的硬件事件异步通知 CPU；**异常 (Exception)**: 由当前指令或执行结果同步触发的控制转移。
- **引导 (Boot)**: 机器上电后由固件和引导程序逐步装入内核、建立最小运行环境并交出控制权的过程。

这些词后文若出现缩写，均沿用这里的含义；具体机制由 OS-01/02 至 OS-06/07 分册展开。

Owner / Stop Boundary

本册完整拥有：OS 基本概念、发展逻辑、408 四特征、程序链接与装入的入口语义、运行时映像的基础区分、OS 结构、系统引导、机器级虚拟化，以及这些概念与后续 Topic 的导航关系。

以下内容只给**最小接口**，不重新拥有内部机制：

- user/kernel mode、system call、exception、interrupt 的控制权细节 → OS-01/02；
- 同步互斥、PV、管程、死锁 → OS-03；
- 地址翻译、页表、缺页、页置换、COW/mmap → OS-04；
- driver、DMA、I/O request、buffering、device scheduling → OS-05；
- pathname、fd/OFD、inode、文件分配与一致性 → OS-06/07；
- 特权级、MMU、IRQ controller 的硬件实现 → CPU × OS Bridge。

目录

1.1	从一条指令开始：裸机模型看起来很简单	3
1.2	Abstraction：先把“怎么做”隐藏成“我能做什么”	3
1.3	Virtualization：把有限资源变成多个可管理的逻辑实例	4
1.4	Protection / Isolation：虚拟化必须带权限边界	4
1.5	Resource Management：虚拟化之后还必须决定“谁先用、用多少”	5
1.6	OS 的功能与服务：不要再背“处理机/存储器/文件/设备管理”四张孤立卡片	5
2	第二章：OSTEP 三条主线与 408 四特征——两套坐标，不要硬凑	6
2.1	OSTEP 的 Three Easy Pieces 是三类“系统问题”	6
2.2	408 四特征描述的是 OS/多道环境呈现出来的性质	6
2.3	四特征的因果关系比并列背诵更重要	6
2.4	虚拟化按什么维度复用？异步又逼出什么正确性底线？	7
3	第三章：操作系统为什么一步步变成今天这样——用失败推动历史	7
3.1	历史不是年代记，而是“旧运行方式的瓶颈”	7
3.2	人工操作：机器时间被人工交接浪费	7
3.3	批处理与单道：减少人工切换，但 I/O 一来 CPU 仍空闲	8
3.4	多道程序设计：用并发覆盖等待	8
3.5	分时系统：吞吐率不够，人还需要“快响应”	8
3.6	实时系统：平均快还不够，必须满足时限约束	8
4	第四章：程序运行环境——Program、Executable、Process Image 与 Process 不是同一对象	9
4.1	最容易缺的一条主链：静态描述怎样变成运行实体	9
4.2	链接到底在解决什么？——多个模块怎样成为一个可执行整体	9
4.3	同一个 ELF 为什么有 Section View 与 Segment View？	10
4.4	静态链接与动态链接：绑定发生在什么时候？	11
4.5	三种经典装入方式：地址分别在何时确定	12
4.6	装入不是“把整个程序文件一次性复制进物理内存”	12
4.7	把所有“地址”排成一条生命周期：408 最重要的统一图	13
4.8	运行时内存映像：知道有哪些区域，但不要在这里讲页表	15
5	第五章：程序怎样跨过权限边界——本册只建立运行环境入口	16
5.1	User Mode 与 Kernel Mode：为什么必须有权限层级	16
5.2	特权指令、敏感指令与保护闭环	17
5.3	为什么需要系统调用？	17
5.4	参数怎样跨过保护边界	17

5.5	System Call、Exception、Interrupt 只在此处建立分类接口	18
6	第六章：操作系统结构——哪些能力必须放在特权核心里	18
6.1	不要背优缺点表，先建立比较轴	18
6.2	宏内核 (Monolithic Kernel)：大量核心服务共处内核地址空间	19
6.3	模块化：解决“一个大内核怎样组织和扩展”	19
6.4	分层结构：限制跨层依赖	20
6.5	微内核 (Microkernel)：把高层服务移出特权核心	20
6.6	外核 (Exokernel)：更进一步，把“保护”和“资源管理策略”分开	20
7	第七章：系统引导——OS 自己也必须先被加载和获得控制权	21
7.1	Bootstrap Paradox：还没有 OS 时，谁来加载 OS？	21
7.2	Firmware：先把平台带到可加载下一阶段的状态	21
7.3	Legacy BIOS 与 UEFI：同一母问题的两种查找协议	21
7.4	Bootloader：找到内核、准备参数、移交控制	22
7.5	Kernel 启动之后还没有“完整用户环境”	22
8	第八章：虚拟机——OS 虚拟化资源，不等于 VMM 虚拟整台机器	22
8.1	两种 virtualization 必须分层	22
8.2	VMM 的母问题：多个 OS 怎样安全共享同一台物理机器？	23
8.3	CPU 虚拟化：怎样让 Guest 的敏感操作可被 VMM 观察	23
8.4	内存虚拟化：Guest 物理地址仍不是机器物理地址	24
8.5	I/O 虚拟化：兼容、批量与直通的陷入频率权衡	24
8.6	Type 1 / Type 2 只作为部署结构理解	24
9	第九章：一张总图把 OS-00 挂到后续所有 Topic	25
10	第十章：最小调用接口——先定位层，再调用 Owner	25
10.1	忘掉名词时怎样重新推出来？	25
	参考与工程边界来源	26

1 第一章：从裸机到可用机器——程序需要哪些服务

1.1 从一条指令开始：裸机模型看起来很简单

若只从 CPU 的视角观察，一个正在运行的程序似乎只是不断重复：

Fetch → Decode → Execute

程序读写内存、执行算术、分支和函数调用，直到结束。

问题在于：这只描述了单个程序在理想机器上的局部执行，不描述真实系统怎样让很多程序安全、方便、有效地共享同一套硬件。

把真实机器放回来，立刻出现四组矛盾：

1. **资源有限**：CPU 核心、内存、设备和持久存储都有限；
2. **多个程序竞争**：多个程序同时想用 CPU、RAM 和设备；
3. **直接访问危险**：任何程序若可任意修改页表、设备控制寄存器或别人的内存，隔离立即失效；
4. **硬件接口复杂且异步**：设备速度不同，完成事件可能在程序执行的任意时刻到来。

因此程序不直接操作某块裸硬件，而是通过操作系统提供的接口使用 CPU、内存、文件和设备；内核同时检查权限并隔离不同程序。

第一条生成链：操作系统的双视角 (Twin Viewpoint)

OS = Resource Manager + Extended Machine / Interface

分析操作系统的任何功能时，固定从双重视角看：

1. **自下而上：资源管理者 (Resource Manager)**：面对物理硬件（CPU、内存、设备、磁盘），负责分配、复用、隔离与并发管控；
2. **自上而下：扩展机与抽象层 (Extended Machine / Interface)**：面向应用程序，隐藏底层接口细节，把粗糙的物理硬件重新解释为优雅的逻辑抽象（进程、虚拟内存、文件、系统调用）。

1.2 Abstraction：先把“怎么做”隐藏成“我能做什么”

硬盘提供块、扇区和控制命令，应用却按 `/home/a.txt` 访问文件。物理内存是一串字节位置，程序却使用自己的虚拟地址。CPU 数量有限，调度器则让多个执行流轮流获得处理器时间。

因此第一层不是虚拟化，而是**抽象 (Abstraction)**：

Complex Physical Interface → Stable Logical Interface

典型例子：

物理资源	程序希望看到的抽象	后续 Owner
CPU	process / thread、可运行执行流	OS-01/02
内存	address space、可保护的虚拟地址	OS-04
存储设备	file / directory / namespace	OS-06/07
I/O 设备	统一或受控的 I/O interface	OS-05

Abstraction $\neq$ Virtualization

抽象回答：“程序应该通过什么对象和接口使用资源？”

虚拟化回答：“怎样让有限物理资源表现得像更丰富、可独立使用的逻辑资源？”

两者经常一起出现，但不能当作同义词。

1.3 Virtualization：把有限资源变成多个可管理的逻辑实例

OSTEP 将 virtualization 作为 OS 的核心组织原则之一：OS 接收 CPU、memory、disk 等物理资源，把它们转化成更通用、更方便使用的虚拟形式，同时通过 mechanism 与 policy 决定怎样共享这些资源。

最直观的例子是 CPU：只有少量 CPU 核心，却能让大量程序都表现为“正在运行”。这不是凭空创造算力，而是：

$$\text{Physical CPU Time} \xrightarrow{\text{Multiplexing} + \text{State Preservation}} \text{Many Logical Execution Streams}$$

内存同理：

$$\text{Physical Memory} \xrightarrow{\text{Mapping} + \text{Protection}} \text{Per-process Address Spaces}$$

题目信号

看到“多个进程都认为自己从同一虚拟地址开始”“单核却有多个程序同时运行”“逻辑资源数量大于物理资源数量”时，第一动作不是背算法，而是问：

物理资源是什么？虚拟对象是什么？谁负责映射和复用？

1.4 Protection / Isolation：虚拟化必须带权限边界

如果只是“共享”，没有隔离，一个程序就能破坏另一个程序或内核。因此：

$$\text{Sharing without Protection} \Rightarrow \text{Unsafe System}$$

OS 必须控制：

- 哪些指令普通程序不能直接执行；
- 哪些地址可以访问；

- 哪些资源对象具有访问权限；
- 哪些状态只能由内核修改。

Protection 是后续 user/kernel mode、memory protection、file permission、capability 等机制共享的上位问题。本册只建立这个目标，不在这里展开硬件特权级和具体权限实现。

1.5 Resource Management：虚拟化之后还必须决定 “谁先用、用多少”

当多个程序竞争同一资源时，仅有 mechanism 不够，还需要 policy：

Mechanism: 能不能做? Policy: 选择谁做?

例如：

- CPU 可以被切换，这是 mechanism；下一位运行谁，是 scheduling policy；
- 页面可以被换出，这是 mechanism；淘汰哪一页，是 replacement policy；
- 磁盘请求可以排队，这是 mechanism；下一请求选谁，是 device scheduling policy。

这解释了 OS 为什么也被称为 **resource manager**。

1.6 OS 的功能与服务：不要再背 “处理机/存储器/文件/设备管理” 四张孤立卡片

传统教材常把 OS 功能列成处理机管理、存储器管理、文件管理、设备管理，再补用户接口。更好的挂法是按**资源域 + 对外服务**理解：

资源/对象域	OS 必须解决的母问题	后续 Owner
CPU / execution	谁能运行、何时运行、怎样保存/恢复执行上下文	OS-01/02
shared state	多个执行流交错时怎样保持正确性与进展	OS-03
memory	地址空间、分配、保护、驻留和回收怎样管理	OS-04
I/O device	请求怎样提交、等待、完成并与设备速度差异解耦	OS-05
persistent storage	名字、对象、内容、空间和崩溃后一致性怎样维持	OS-06/07

从应用视角，OS 还必须提供**可调用接口**：程序执行、I/O、文件/目录操作、通信、错误处理、资源分配、保护等最终通过 API / system-call boundary 暴露。这里要保持：

Library API ≠ System Call ≠ Kernel Internal Function

一个库函数可能完全在用户态完成，也可能封装一个或多个 system calls；system call 是受控进入内核服务的接口语义。

功能题第一动作

题目问“操作系统功能/服务”时，先问它在管理哪个**资源域**、向谁提供什么**可观察接口**，再把它送到后续 Owner。不要把“管理功能”和“用户接口”混成同一层。

2 第二章：OSTEP 三条主线与 408 四特征——两套坐标，不要硬凑

2.1 OSTEP 的 Three Easy Pieces 是三类“系统问题”

OSTEP 用三条主线组织操作系统：

Virtualization + Concurrency + Persistence

它们分别追问：

- 1. **Virtualization**：怎样把 CPU、memory 等物理资源变成可共享的逻辑资源？
- 2. **Concurrency**：多个活动同时推进时，交错和共享状态怎样保持正确？
- 3. **Persistence**：数据怎样跨越进程生命周期和断电继续存在？

它们是**课程组织轴 / 问题轴**，不是 408 四特征的英文翻译。

2.2 408 四特征描述的是 OS/多道环境呈现出来的性质

特征	判断依据	容易混淆的概念
并发	多个事件在一段时间内交替推进	并发 ≠ 并行；单核也可并发
共享	多个执行实体共同使用有限资源	互斥共享与同时共享不同
虚拟	物理实体被变换成逻辑上的多个/更易用实体	虚拟 ≠ “不存在”
异步	进程推进速度和完成时刻具有不可预知性	异步来自调度、竞争、设备完成等不确定交错

2.3 四特征的因果关系比并列背诵更重要

不要把四个词当四张卡片。更好的生成关系是：

有限资源被共享 ⇒ 多个活动并发推进 ⇒ 执行次序和进度呈异步性

同时，为了让共享更方便、安全，OS 使用虚拟化：

Physical Sharing $\xrightarrow{\text{Virtualization + Protection}}$ Logical Isolation / Managed Sharing

边界辨析

- **并发 ≠ 并行**：并发强调时间段内多个活动都在推进；并行强调同一时刻同时执行。
- **共享是并发的资源条件之一**：若完全没有资源关系，很多并发冲突就不会出现。
- **异步不是“使用异步 I/O”** 它是更一般的推进次序不确定性。
- **虚拟不是模拟器专属术语**：CPU 时间复用、地址空间、文件抽象都体现广义虚拟化思想。

2.4 虚拟化按什么维度复用？异步又逼出什么正确性底线？

在 408 的经典分类中，“把一个物理实体变成多个逻辑对应物”还可以继续追问：**被切分的是时间，还是空间？**

复用方式	典型对象	生成解释
时分复用	CPU、SPOOLing 所形成的逻辑设备	不同使用者在不同时间片取得同一物理资源；“逻辑上各有一份”来自快速切换与状态保存
空分复用	虚拟存储器、磁盘分区	不同使用者同时占有物理资源的不同空间部分；逻辑边界来自地址/区域划分与保护

这是一种**教材分析轴**，不是说所有工程机制只能贴一个标签。例如虚拟内存的容量幻觉还依赖换入换出等时间行为；具体页生命周期由 OS-04 拥有。对 CPU 而言，单核不能把执行单元“空间切成两半”同时执行两个进程，因此经典的 CPU 虚拟化主要表现为时分复用。

异步也不只意味着“速度忽快忽慢”。它提出的正确性底线是：只要外部输入和同步约束相同，系统就不能因为某次偶然交错而破坏应有的结果。若共享状态未受保护，不同调度交错会导致结果不可再现；如何用互斥、同步与原子机制守住这条底线转交 OS-03。

3 第三章：操作系统为什么一步步变成今天这样——用失败推动历史

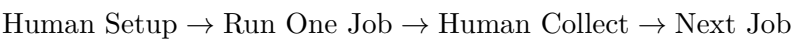
3.1 历史不是年代记，而是“旧运行方式的瓶颈”

OS 的发展可以用两个不断变化的目标驱动：



3.2 人工操作：机器时间被人工交接浪费

早期模式近似：



失败点：CPU 很昂贵，但大量时间在等待人工装入、切换和准备。

由此推动**自动作业接续**。

3.3 批处理与单道：减少人工切换，但 I/O 一来 CPU 仍空闲

单道批处理解决“作业自动连续执行”，但若一个作业等待慢速 I/O：

Job waits for I/O $\Rightarrow$ CPU idle

因此下一步自然不是“换一种名字”，而是：内存里同时保留多个作业，当前作业等待时让另一个运行。

3.4 多道程序设计：用并发覆盖等待

One Job Waits $\Rightarrow$ Run Another Ready Job

这直接带来现代 OS 的一系列问题：

- 多个作业如何共存于内存？
- CPU 下一步给谁？
- 一个程序如何不能破坏另一个程序？
- 多个程序共享设备时如何排队？

换句话说，多道程序设计不是孤立历史名词，而是进程、调度、内存保护、资源管理问题集体出现的原因之一。

这里还要补一条软硬件闭环：只有 CPU 能在设备工作期间继续执行，并在设备完成时重新获得事件通知，计算与 I/O 才可能重叠。教材语境常把中断 + 通道/独立 I/O 控制能力概括为多道程序设计的重要硬件基础；现代实现中的 interrupt controller、DMA 与设备队列细节分别转交 CPU $\times$ OS Bridge 和 OS-05。

3.5 分时系统：吞吐率不够，人还需要“快响应”

批处理更关心单位时间完成多少作业；交互用户更关心：

我发出请求后多久看到反馈？

于是 CPU 被更细粒度地在多个交互任务之间切换，目标从纯 utilization/throughput 扩展到 response time 与 fairness。

3.6 实时系统：平均快还不够，必须满足时限约束

实时系统的关键不是“运算速度特别快”，而是：

Correct Result + Timing Constraint

因此平均响应时间可能不是主要指标；最坏情况、deadline、可预测性变得重要。

历史题统一做法

题目问某种 OS 形态时，不先背定义，先问：

1. 上一代系统浪费在哪里？
2. 新系统主要优化 utilization、throughput、response 还是 deadline？
3. 为了这个目标，引入了什么新的共享/调度关系？

4 第四章：程序运行环境——Program、Executable、Process Image 与 Process 不是同一对象

4.1 最容易缺的一条主链：静态描述怎样变成运行实体

一段源代码离 CPU 执行还有多层变换：

Source → Object File → Executable / Shared Object → Process Image → Running Process

箭头含义分别是：编译/汇编产生目标文件；链接组合目标文件和符号关系；装入根据可执行文件描述建立运行时映像；随后 OS 建立可管理的执行实体并把控制交给程序入口。

四个对象不能混

- **Program / Executable**：静态文件；
- **Process Image**：某次执行所需的运行时地址空间内容与初始布局；
- **Process**：OS 管理的动态执行实体；
- **Address Space**：该执行上下文看到的地址集合及其映射语义，完整机制由 OS-04 拥有。

4.2 链接到底在解决什么？——多个模块怎样成为一个可执行整体

独立编译的根本代价是：每个模块只知道自己的局部布局，却不知道其他模块最终在哪里。例如 `main.o` 调用另一个目标文件中的 `foo`，汇编器可以知道“这里需要调用 `foo`”，却未必知道 `foo` 在最终映像中的位置。于是链接器必须解决两个不同问题：

Symbol Resolution + Relocation

Symbol Resolution（符号解析） 回答：这个引用究竟绑定到哪个定义？例如 `foo` 应关联哪个函数定义。它建立的是“名字/引用 $\mapsto$ 定义”的关系。

Relocation（重定位） 回答：定义和各模块位置已经确定后，哪些机器码或数据字段必须怎样改写？它处理的是“引用位置 + relocation type + 最终地址布局 $\mapsto$ 修补值”。

因此二者不能合并成“链接就是改地址”：若连 `foo` 指向谁都还没有决定，就没有资格计算最终修补值；反过来，知道 `foo` 对应哪个定义，也不等于所有引用位已经编码正确。

为什么目标文件需要 Symbol Table 与 Relocation Records?

可重定位目标文件本质上在保存两类尚未完成但信息已足够描述的工作：

- Symbol table 记录本模块定义/引用了哪些可链接对象，以及它们当前属于哪里；
- Relocation records 记录“哪个位置以后必须修、按什么规则修、它依赖哪个符号”。

链接器拿到多个模块后，才同时获得全局定义集合与最终布局，从而把这份待办清单兑现。

本册只保留链接层面的语义。若题目追问逻辑/虚拟地址怎样在运行期映射到物理地址，立即切到 OS-04; **link-time relocation $\neq$ run-time address translation**。

4.3 同一个 ELF 为什么有 Section View 与 Segment View?

先从学生最容易遇到的困惑开始：为什么同一个可执行文件，链接器和装入器看到的“分块方式”不一样？

假设程序里只有三样东西：一段函数代码、一个已经初始化的全局变量和一个未显式初始化的全局变量。编译、汇编之后，这些内容需要被保存进目标文件；链接时还要继续解决符号和重定位。因此链接器更关心“这些内容各自是什么、该和谁合并、哪些地方还要修改”。这就是 Section 视角。

但程序真正运行时，OS/Loader 不需要逐个理解“这是 .text、这是 .data、这是某张符号表”。它更关心的是：文件中的哪一段字节要映射到哪个 VA 范围、需要什么权限、内存中要占多大范围。这就是 Segment 视角。

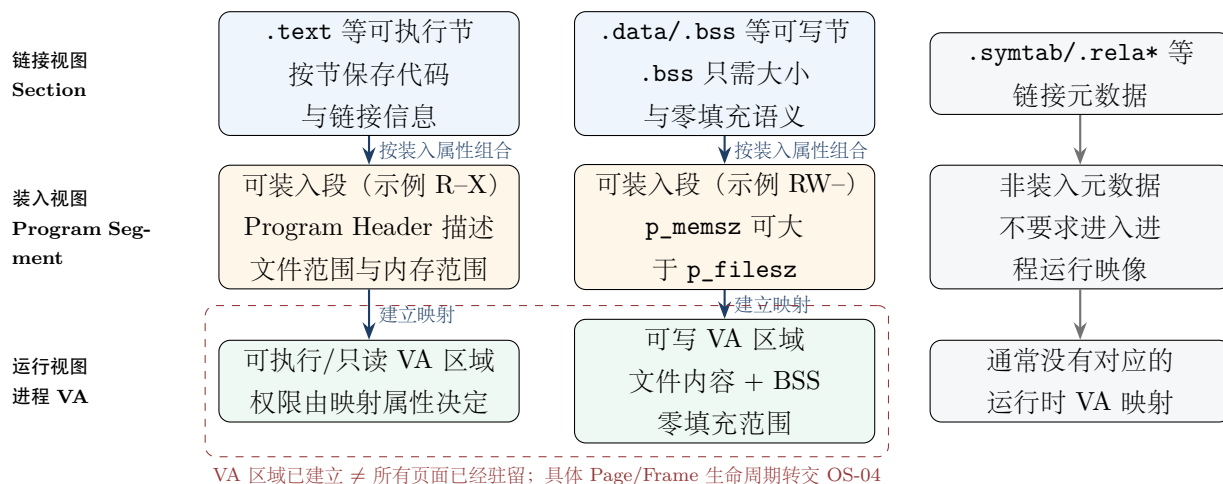
所以，Section 和 Segment 不是两套互相竞争的分类，而是同一份文件为两个不同阶段提供的两种视图：

Linking View: Sections	$\neq$	Loading View: Segments
------------------------	--------	------------------------

Section (节) 主要服务于链接和静态组织，例如代码、数据、符号表、重定位信息等；链接器关心的是“哪些内容应合并、哪些符号要解析、哪些位置要修补”。**Segment (段, program segment)** 主要服务于程序装入；program header 告诉装入路径“文件中的哪些范围应映射成什么样的运行时内存区域、具有什么权限”。多个 section 可以按装入属性组合进同一个 loadable segment，因此不能把 .text/.data 等 section 名机械等同于运行时的每个 segment。

Section $\neq$ Segment，也不要吧中文“段”全都当一个概念

教材中“代码段/数据段”“分段存储管理的 segment”“ELF program segment”会共享“段”字，但 Owner 和判据不同。看到 ELF 时先问这是链接器的 section 组织，还是 loader 的 program segment 映射？看到内存管理时再问是否讨论 OS-04 的 segmentation 地址机制。



怎样一步一步读这张图？

不要从左到右扫名词，按下面三步读：

1. **先看第一行：**这是“文件里怎样组织内容”。`.text/.data/.bss/.symtab` 等名字首先属于静态文件组织。它们回答“这是什么内容”，不是“现在已经在 RAM 的哪里”。
2. **再看第二行：**Loader 不按 section 名逐个搬运，而是按照 program header 给出的 segment 描述决定“文件哪些范围要映射、映射后内存多大、权限是什么”。因此多个 section 可以被组合进同一个可装入 segment。
3. **最后看第三行：**segment 被用来建立进程的 VA 区域。到这里才进入某次执行的地址空间，但仍然只说明“这个 VA 范围合法、由什么对象支撑、具有什么权限”，**不表示其中每一页都已经在物理内存。**

因此这张图真正要记住的是：

文件组织 ≠ 装入组织 ≠ 当前物理驻留状态

上图只是关系示意，不承诺任何具体 ELF 都恰好只有两个 `PT_LOAD`，也不承诺某个 section 永远属于固定权限的 segment。

用 BSS 检查你是不是真的看懂了

假设程序中有：

```
int counter;
```

它没有显式初值。目标文件/可执行文件没有必要真的存一大片 0，只需要记录“运行时这里需要这么多字节，并且初始为 0”。因此可能出现：**文件中占用很少，但进程 VA 中必须留出实际范围。**这正是 `p_memsz > p_filesz` 一类现象背后的直觉。

如果看到 `.bss` 就说“它已经作为一块全 0 数据存进可执行文件并被整块搬进 RAM”，说明仍然把**文件布局、VA 布局、物理驻留混在了一起。**

4.4 静态链接与动态链接：绑定发生在什么时候？

可用一个统一问题理解：

Dependency is bound when?

方式	绑定时机	主要权衡
静态链接	生成可执行文件时	运行依赖少，但可执行文件更大、共享库代码复用差
装入时动态链接	程序开始建立运行映像时	多程序可共享库，启动时需完成依赖处理
运行时动态链接	程序运行过程中按需装入/绑定	更灵活，但运行时路径和失败情况更复杂

ELF 的动态链接模型进一步体现了这一点：可执行文件可指定 `program interpreter`，由动态链接器参与把 `executable` 和 `shared objects` 组合成完整 `process image`，再把控制交给应用程序。

4.5 三种经典装入方式：地址分别在何时确定

408 传统模型中的装入方式也不要孤立背。统一问：

程序中地址相关引用在什么时候获得足以执行的运行位置解释？

这里的“地址绑定”是教材抽象，不应理解成现代分页系统必须把每个 VA 提前绑定到永久 PA。它想比较的是：地址修补工作是在程序生成前就结束、在装入时一次性完成，还是保留到运行时由硬件翻译。

装入方式	地址确定时机	机制与边界
绝对装入	编译/生成程序时就已知最终绝对地址	只适合运行位置预先确定的简单环境，灵活性最低
可重定位装入（静态重定位）	装入时一次性根据实际起始位置修改地址相关项	装入后通常难以再次移动；“重定位”发生在 <code>load time</code>
动态运行时装入（动态重定位）	程序运行期间保留逻辑/相对地址，访问时借助硬件动态形成实际地址	支持更灵活的移动与虚拟存储；具体 MMU/页表机制归 OS-04

Loading classification ≠ Linking classification

“静态链接 / 装入时动态链接 / 运行时动态链接”讨论模块之间的依赖何时绑定；
“绝对装入 / 可重定位装入 / 动态运行时装入”讨论程序地址何时绑定到运行位置。
两套分类使用了相似的“静态/动态/运行时”词汇，但回答的是不同问题。

4.6 装入不是“把整个程序文件一次性复制进物理内存”

这是一个非常重要的边界。ELF 规范把 `executable/shared object` 看作程序的静态表示，执行时系统依据 `program headers` 创建动态 `process image`。现代系统可以先建立 VA range 与 `executable/shared object` 的映射关系，具体物理页等真正访问时才通过 `demand paging` 获得。

因此：

Load / Map a Program ≠ All Pages Immediately Resident

Loader/exec 路径真正需要建立的是运行资格：新的地址空间描述、映射、权限、初始用户栈/参数以及入口状态。内核必须参与建立受保护映射；但现代动态链接器本身也可以作为用户空间程序参与依赖解析和映射请求，所以不要把“loader”想成一个永远完全位于内核中的单一函数。

这句话只在 OS-00 建立概念边界；为什么可以延迟、页表如何表示、Page Fault 如何修复全部属于 OS-04。

4.7 把所有“地址”排成一条生命周期：408 最重要的统一图

这一部分不要从“逻辑地址、虚拟地址、物理地址”三个定义开始背。先问一个更具体的问题：如果 main.c 中调用了另一个文件里的 foo()，CPU 最后到底怎样找到 foo 的那条机器指令？

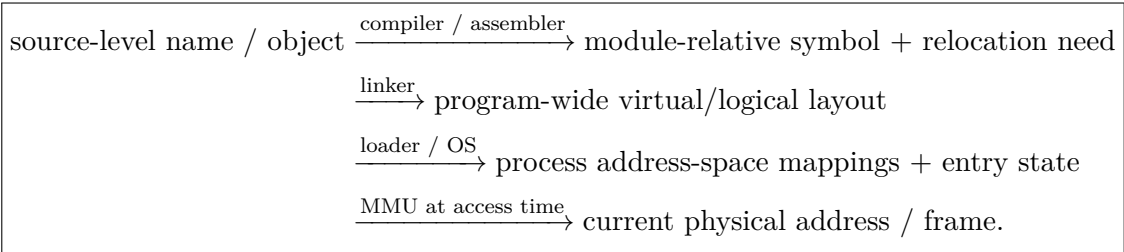
答案不是某一步突然“算出最终地址”，而是每一阶段补上一部分之前不知道的信息：

- 1. 编译/汇编：知道这里有一次对 foo 的引用，也知道这条机器指令中哪个字段以后可能要修；但如果 foo 定义在另一个目标文件里，此时通常还不知道它在最终程序中的位置。
- 2. 链接：把多个目标文件放在一起后，终于知道“这个 foo 引用绑定到哪个定义”，也知道各模块在整个程序里的布局，于是可以计算相对位移、地址值，或者留下动态链接阶段继续处理的重定位信息。
- 3. 装入：可执行文件现在要真正运行。OS/Loader 根据 program headers 给这次进程建立 VA 区域、权限、入口 PC、栈等。此时谈的是这一次执行看到的虚拟地址空间。
- 4. 真正取指：CPU 用某个 VA 取 foo 的机器指令时，MMU 才根据当前页表把这个 VA 翻译成当前 PA；若页面尚未驻留，还可能先发生 Page Fault，再由 OS 补齐映射后重试。

所以真正的主线不是“一个地址被不断改写”，而是：

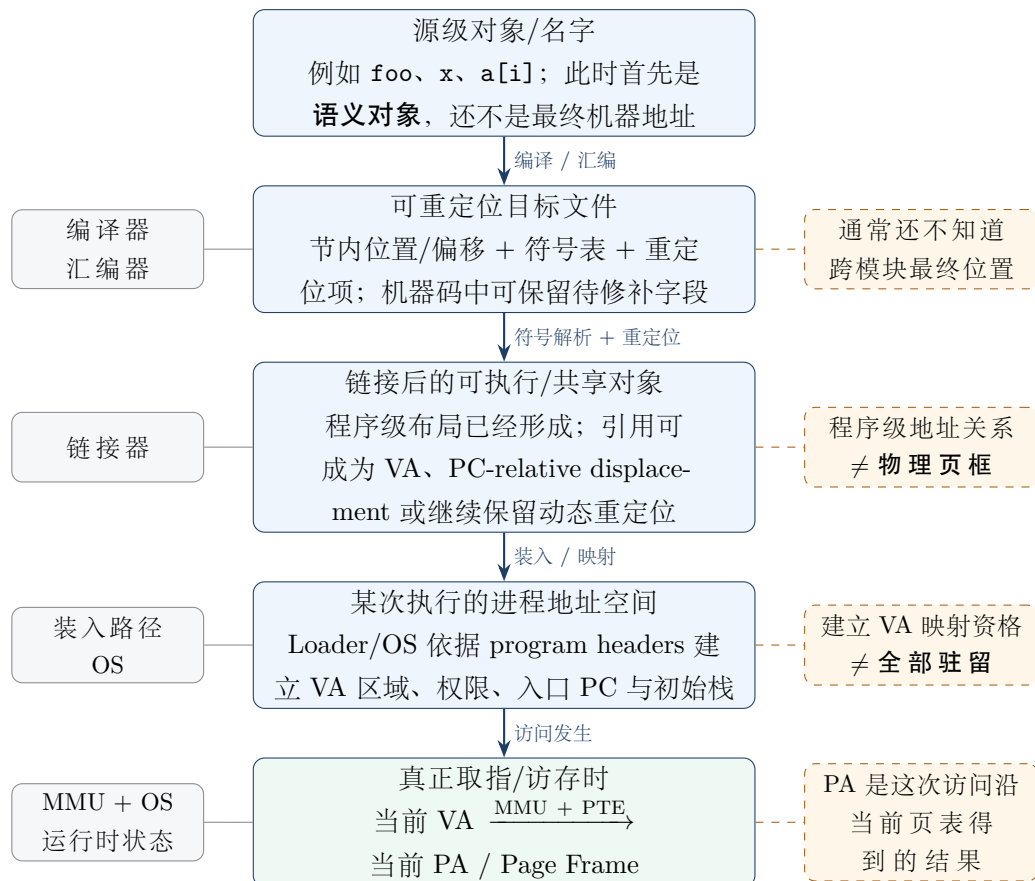
引用关系逐步获得更多上下文，直到一次真实访存能够落到物理内存

稳定地按信息成熟度追踪：



这不是说每种机器都恰好产生四种数值不同的地址，而是规定谁在什么时候知道什么：

- 编译/汇编阶段可以生成位置无关引用、模块内偏移或 relocation placeholder，但通常不知道其他模块最终布局；
- 链接器拥有多个目标文件的布局，因此能形成完整程序级地址关系并修补需要的引用；
- Loader/OS 依据 executable 的装入描述建立某次执行的虚拟地址空间，动态库还可能在这一阶段继续完成依赖绑定；
- MMU 只在真正执行取指/访存时，把当前 VA 按页表等运行时状态翻译成 PA。物理页框属于运行时资源，编译器和普通静态链接器不能预先替 OS 决定。



怎样一步一步读地址生命周期图？

每经过一层，只回答三个问题：现在已经知道什么？还不知道什么？下一层为什么有必要？

- **目标文件层**：已经有机器码、符号和“哪里以后要修”的记录；还不知道完整跨模块布局，所以需要 Linker。
- **可执行文件层**：已经有程序级布局；还没有某次运行对应的进程地址空间，所以需要 Loader/OS。
- **进程 VA 层**：已经知道这个进程可以使用哪些 VA 范围；还不能由此推出某一 VA 此刻对应哪个物理页框，所以需要运行时地址翻译。
- **PA 层**：只针对这一次实际取指/访存，结合当前页表得到物理位置。页表状态以后可以改变，所以 PA 不是“编译时永久写死的地址”。

不要寻找一个贯穿全生命周期的“同一个地址数值”

真正稳定的不变量是**同一个程序对象/引用的语义关系被逐步具体化**，而不是某个数值从编译一路原样传到物理内存。位置无关代码、地址空间布局随机化、动态链接、分页都会让具体数值或绑定时机变化；408 做题应追踪“当前处于哪一层、当前谁有足够信息、当前输出的地址属于哪个地址空间”。

用一个 `foo()` 调用完整跑一遍

假设 `main.o` 中有一条调用 `foo()` 的指令，而 `foo` 定义在 `util.o`。

1. 汇编 `main.o` 时，可以留下“这里引用符号 `foo`，以后按某种 relocation type 修这个字段”的记录；
2. Linker 找到 `util.o` 中 `foo` 的定义，安排两边布局，然后把调用字段修成正确的程序级地址/相对位移，或保留给动态链接器继续处理；
3. 程序执行时，Loader/OS 把包含这段代码的可执行 segment 映射到进程的某个 VA 区域；
4. PC 最终来到调用目标附近，CPU 以 VA 取指；MMU 查询 TLB/页表得到 PA；若该代码页当前不在内存，则先触发缺页，OS 把页面准备好后再重试同一次取指。

这四步分别回答四个不同问题：**引用谁？程序里放哪？这次进程看到哪？此刻 RAM 在哪？**一旦把四问分开，链接重定位和运行时地址翻译就不容易再混。

四个高频问法分别找谁

- “`foo` 到底指哪个函数？” $\Rightarrow$ Symbol Resolution；
- “这条 `call/load` 中的地址字段最终填什么？” $\Rightarrow$ Relocation；
- “这个可执行文件怎样形成进程的 `text/data/stack/mapping`？” $\Rightarrow$ Loading / Process Image；
- “CPU 此刻访问 `VA=...` 最终落到哪个页框？” $\Rightarrow$ OS-04 / CO-07 的 Address Translation。

4.8 运行时内存映像：知道有哪些区域，但不要在这里讲页表

经典进程映像可以按语义识别为：

- `code / text`：程序指令；
- `read-only data`：只读常量；
- `initialized data`：已初始化全局/静态数据；
- `BSS`：未显式初始化的静态存储对象，文件中不必为其保存等量数据；
- `heap`：运行时动态增长的数据区域；
- `stack`：函数调用、局部状态等运行时结构；
- `shared libraries / mapped regions`：共享对象与其他映射区域。

这些区域不能只按“从低地址到高地址”背名称，还要同时追踪**内容来源、文件占用、访问权限与增长方式**：

区域	内容从哪里来	可执行文件中是否保存等量内容	典型权限/变化
Text / Code	编译产生的机器指令	是	Read + Execute, 通常固定且可共享
Read-only Data	字面量、只读常量	是	Read, 通常固定
Initialized Data	有初值的全局/静态对象	是, 必须保存初值	Read + Write, 大小通常固定
BSS	无显式初值的全局/静态对象	否, 只需记录长度, 装入时按协议清零	Read + Write, 大小通常固定
Heap	运行时 allocator / brk/mmap 管理的对象区	否	Read + Write, 按需增长; allocator 与物理页分配不是同一层
Mapped Region	shared object、file-backed 或 anonymous mapping	映射引用对象, 不等于把内容复制进 executable	权限由映射属性决定
Stack	调用约定、局部状态、返回链和保存的寄存器	否	Read + Write, 随调用动态变化

其中 BSS 最能暴露“文件布局不等于运行映像”：大量零初始化对象在 executable 中可以压缩成“长度 + 零填充语义”，但运行时地址范围必须真实存在。类似地，映像中出现一段映射区域，只说明 VA range 与某个 backing object/anonymous source 建立了关系，不保证其全部页面已经驻留。

教材常把用户空间与内核空间画在同一张进程地址空间图中：多个进程的 kernel range 可以映射到相同内核物理对象，并以页级权限阻止用户态访问，从而使受控入口后的内核代码立即可达。现代系统可能为缓解侧信道风险采用更强的页表隔离；这属于实现扩展，不改变考试主干：

Mapped in an Address Space ≠ Accessible in Current Privilege Mode

考试第一动作

题目出现“程序装入、链接、动态库、运行时内存映像”时，先定位题目究竟问的是哪一层：

File Composition? Runtime Image? Address Translation?

前两者 OS-00；最后一个 OS-04。

5 第五章：程序怎样跨过权限边界——本册只建立运行环境入口

5.1 User Mode 与 Kernel Mode：为什么必须有权限层级

若应用程序可以直接执行所有敏感操作，Protection 目标无法成立。因此 CPU 提供不同权限状态，OS 使用它们建立：

Unprivileged Application ↔ Privileged Kernel

这里只需要记住：**CPU mode 是硬件执行权限状态，不等于进程状态。**

Ready/Running/Blocked 是任务调度状态；User/Kernel 是当前 CPU 权限状态。两套状态轴不能混。

5.2 特权指令、敏感指令与保护闭环

“内核态权限更高”必须落实到硬件判据。**特权指令**是指在低权限状态执行时会被硬件拒绝并触发同步异常的指令；典型对象包括修改中断开关、定时器、地址翻译/保护寄存器、设备控制状态以及受保护返回状态。**陷入/访管指令反而必须允许用户态执行**，否则应用无法发出系统调用请求；它的受控性来自入口由硬件和内核表选择，而不是让用户任意跳到内核地址。

保护过程包括：

User attempts a privileged operation → operation is blocked by hardware
→ synchronous exception → kernel policy

若只说“用户态不能执行”而不说明**指令未生效、硬件上交控制、内核决定处置**，就没有解释隔离如何成立。

虚拟化语境还需要另一条轴：**敏感指令**是会读取/修改关键机器状态，或其行为随运行模式而变化的指令。两者判据不同：

Privileged? 看低权限执行是否 trap; Sensitive? 看是否触碰虚拟机关键状态。

“敏感但不特权”不会自动把控制交给 VMM，是后文二进制翻译、半虚拟化与硬件辅助虚拟化产生的根因。

5.3 为什么需要系统调用？

应用不能直接做特权操作，但又必须访问文件、设备、创建进程等内核服务，于是需要受控入口：

User Request → Controlled Kernel Entry → Service → Return

system call 的核心不是“函数调用名字很特殊”，而是**跨越保护边界，请求内核代为执行受保护操作**。

系统调用可以按管理对象分为进程/线程、文件、设备、内存、通信与信息维护等类；分类数量不是重点，判断依据是：**操作是否需要访问或改变只能由内核拥有的状态与资源**。库函数也不能与 system call 画等号：库函数代码在用户态运行，只有其执行路径最终发出受控陷入时，才进入内核执行系统调用。

5.4 参数怎样跨过保护边界

系统调用号与参数不能靠“把任意内核入口地址交给 CPU”传递，否则用户可以绕过入口校验。稳定接口是**受限的 service number + ABI 规定的参数位置**：

方式	适用场景	必须检查的边界
陷入指令的直接/间接参数	参数极少、指令格式明确支持	指令编码空间有限，接口不易扩展
通用寄存器	参数少、常走快路径	数量与位宽有限，寄存器约定属于 ABI
寄存器指向用户参数块	参数多、结构体或缓冲区	内核必须检查范围、权限、长度和复制时的竞态
约定的栈/内存区	参数个数可变或需要调用约定组织	进入内核可能切换到 kernel stack，不能假定当前内核栈顶下方就是用户参数

因此“参数在用户内存里”只表示调用方给出了一个地址，不表示内核可以无条件解引用：

User Pointer → Validate / Copy → Kernel-owned Representation

这条边界同时解释了为何系统调用入口必须统一、参数必须校验，以及为何大缓冲区通常传地址而不是逐字节塞进寄存器。

5.5 System Call、Exception、Interrupt 只在此处建立分类接口

入口	与当前指令关系	核心含义
System Call	当前程序主动发起、同步	主动请求 OS 服务
Exception	由当前指令执行触发、同步	当前执行出现需要特殊处理的条件
Interrupt	通常来自外部设备、异步	外部事件要求 CPU/OS 响应

具体 trap 保存什么、硬件自动完成哪些动作、何时调度、interrupt controller 如何仲裁，全部切到 OS-01/02 与 CPU×OS Bridge。

6 第六章：操作系统结构——哪些能力必须放在特权核心里

6.1 不要背优缺点表，先建立比较轴

所有 OS 结构问题都可以压缩成：

Where should mechanism, policy and complexity live?

内核结构演进的评价轴：模块独立性

Complexity → Decomposition → Module Boundary → High Cohesion + Low Coupling → Tradeoff

不同内核结构把服务放在不同位置。评价时直接比较以下四组指标：

1. **High Cohesion (高内聚)**：模块内部是否专注完成单一逻辑职责（如虚拟内存管理、进程调度）；
2. **Low Coupling (低耦合)**：模块之间是否通过稳定、受限的接口通信，避免随意访问内部数据结构；
3. **Fault Isolation (故障隔离)**：一个模块（如驱动程序）崩溃，是否会拖垮整个内核；
4. **Boundary Cost (边界成本)**：跨越模块/权限边界时付出的函数调用、内核态切换或 IPC 开销。

思考任何内核架构题时，问五件事：

1. 哪些组件运行在 privileged kernel?
2. 组件之间是直接函数调用还是 IPC/message?
3. 故障隔离边界在哪里?
4. 扩展/替换组件有多容易?
5. 额外抽象或通信带来什么性能成本?

6.2 宏内核 (Monolithic Kernel)：大量核心服务共处内核地址空间

典型模型：

User Apps → Large Privileged Kernel: FS + VM + Network + Drivers + Scheduler → Hardware

优势来自组件之间可以高效直接协作；风险来自大量复杂代码共享高权限和故障域。

注意：“宏内核”不等于“所有代码写成一个源文件”。现代宏内核完全可以高度模块化。

6.3 模块化：解决“一个大内核怎样组织和扩展”

模块化强调：

Well-defined Interfaces + Replaceable / Loadable Components

它与宏内核不是互斥分类。一个系统可以是 monolithic-kernel architecture，同时支持 loadable kernel modules。

因此：

Monolithic vs Microkernel ≠ Modular vs Non-modular

6.4 分层结构：限制跨层依赖

分层思想要求高层使用低层服务：

$$L_n \rightarrow L_{n-1} \rightarrow \cdots \rightarrow L_0$$

优点是接口和责任清晰；代价是现实需求未必天然适合严格层级，额外层次也可能引入路径成本。

6.5 微内核（Microkernel）：把高层服务移出特权核心

微内核的生成问题是：

What is the minimum that must remain privileged?

典型结构：

Applications / User-space Servers $\xrightarrow{IPC}$ Minimal Kernel Mechanisms $\rightarrow$ Hardware

seL4 的官方描述非常适合作为工程边界：微内核只保留控制物理地址空间、interrupt 和 processor time 等最小机制，高层文件、进程、socket 等抽象可以由用户态系统服务构造。

“内核留机制、用户态服务器放策略”可以作为考试中的统一判据，但不能机械理解为“所有策略都绝不进入内核”：真实系统会因性能、安全与可验证性重新划界。基础题中可用三类最小能力检查：低级调度/IPC、低级地址空间机制、interrupt/trap 的前端处理；页面置换选择、用户分类与优先级原则等高层策略则更适合放在服务器侧。

于是 tradeoff 很清楚：

Smaller Privileged TCB / Better Isolation

$\leftrightarrow$

More IPC / Boundary-crossing Cost

教材中的简化请求路径可以把代价“数出来”：整体式内核的一次用户请求通常抽象为 user $\rightarrow$ kernel $\rightarrow$ user 两次边界切换；若微内核服务由用户态服务器完成，则至少要经过 client $\rightarrow$ kernel $\rightarrow$ server $\rightarrow$ kernel $\rightarrow$ client。若服务器继续请求另一个服务器，路径还会增长。这里的“2 / 4 / 8 次”是**教学路径计数**，会随 IPC fast path、调度方式和硬件支持改变；稳定结论是跨地址空间 IPC、调度与数据传递增加了边界成本，而不是某个常数永远成立。

6.6 外核（Exokernel）：更进一步，把“保护”和“资源管理策略”分开

外核思想的核心不是“更小的微内核”，而是：

Kernel protects and multiplexes physical resources; applications choose abstractions/policies

也就是把传统 OS 为所有程序统一规定的高层抽象下放，让 library OS / application 更直接决定资源如何使用。

因此微内核与外核不是同一条“越做越小”的尺寸轴：

Microkernel asks: where does a service implementation run?

Exokernel asks: who constructs the resource abstraction?

微内核把文件系统等服务实现移到用户态服务器，应用仍可看到统一文件抽象；外核则只在物理页框、磁盘块等粒度上分配和保护资源，由与应用同址空间的 **LibOS** 构造文件、地址空间等抽象。

LibOS 让高层抽象调用退化为进程内函数调用，并允许数据库等应用按自身访问模式定制策略；代价是应用复杂度上升、不同 LibOS 之间的兼容与共享变难、内核只能在更低资源粒度保护，以及系统的全局优化能力下降。于是外核的真实交换不是单纯“更快”，而是：

Application-specific Control	↔	Uniform Abstraction / Compatibility / Global Coordination
------------------------------	---	---

408 做题时抓住：**exokernel** 倾向于把资源保护留在内核，把更高层的资源管理和抽象选择交给应用/库。

结构题第一动作

不要一上来背“微内核可靠、宏内核快”。先画 privilege boundary：

哪些服务在内核？哪些在用户态？跨边界靠什么？

然后性能、隔离、可扩展性的结论自然出来。

7 第七章：系统引导——OS 自己也必须先被加载和获得控制权

7.1 Bootstrap Paradox：还没有 OS 时，谁来加载 OS？

OS 能装入应用程序，但开机时 OS 尚未运行。于是必须存在一条比 OS 更早的启动链：

Power On → Firmware → Boot Manager / Bootloader → Kernel → Initial User-space System
--

这就是 bootstrap：使用一个更小、更早可运行的环境，把更完整的系统加载起来。

7.2 Firmware：先把平台带到可加载下一阶段的状态

Firmware 负责平台早期初始化与启动策略。现代 UEFI 模型中，firmware 内的 Boot Manager 会按照 boot configuration 选择并加载 UEFI driver/application，包括 OS boot loader，并把控制传给下一阶段。

因此不要把 BIOS/UEFI 与 bootloader 混成同一个对象：

Firmware ≠ OS Bootloader ≠ Kernel

7.3 Legacy BIOS 与 UEFI：同一母问题的两种查找协议

传统 BIOS/MBR 教材模型依赖“固定位置”逐级扩大可理解的元数据范围：

Reset Vector → Firmware → Disk Sector 0 / MBR → Active Partition / PBR → Kernel or richer Bootloader

经典 MBR 一个 512-byte sector 由三部分构成：446 B bootstrap code、64 B partition table、2 B signature。若每个 partition entry 为 16 B，则 $64/16 = 4$ 个主分区表项；这是格式容量推导，不是“磁

盘天然只能有四个分区”。PBR 位于所选分区的启动区域，它必须借助该分区格式的最小元数据/读盘逻辑继续找到下一阶段，而不是调用尚未运行的完整文件系统服务。

UEFI 让 firmware 承担更多启动工作：firmware 可按配置读取 EFI System Partition 中的 EFI application/bootloader，因而不要求 446 B MBR 代码自行逐扇区查找下一阶段。两种方式仍采用相同的逐级加载过程：

Earlier Stage knows just enough to locate, validate and transfer control to Later Stage

不要把 MBR/PBR 的精确数字外推成所有现代平台的普遍实现。

7.4 Bootloader：找到内核、准备参数、移交控制

不同平台和 OS 的细节差异很大，但抽象职责稳定：

1. 找到可启动的 kernel image；
2. 根据协议把 kernel/必要数据放到合适位置；
3. 准备启动参数和机器状态；
4. 跳转到 kernel entry，把控制权交给内核。

7.5 Kernel 启动之后还没有“完整用户环境”

Kernel 接管机器后仍需初始化核心子系统，并创建/启动第一个用户态管理实体，之后才逐步形成服务、登录环境和普通应用。

教材中的经典进程编号叙述可压缩为：内核初始化阶段先把当前执行上下文纳入内核任务管理，再由内核创建初始用户态管理进程。具体编号与启动实体名称依系统而异；稳定事实是**最初的用户进程不是由一个更早的普通用户进程创建，而是由已经取得控制权的内核启动。**

因此开机过程不是：

Power → OS appears

而是一个控制权逐级交接和环境逐步扩充的过程。

引导题信号

遇到“ROM/firmware/bootloader/kernel 谁先谁后”时，只做一件事：追踪下一阶段代码是谁加载的、控制权交给了谁。

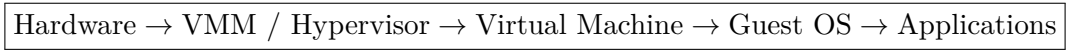
8 第八章：虚拟机——OS 虚拟化资源，不等于 VMM 虚拟整台机器

8.1 两种 virtualization 必须分层

OS 内部的资源虚拟化：

Hardware → Operating System → Processes / Address Spaces / Files

机器级虚拟化：



前者给应用程序提供 OS abstraction；后者给另一个操作系统提供一台近似完整的 virtual hardware machine。

8.2 VMM 的母问题：多个 OS 怎样安全共享同一台物理机器？

VMM 必须虚拟化/仲裁：

- CPU execution；
- memory；
- interrupt / timer；
- virtual devices / I/O；
- isolation between VMs。

它与普通进程隔离相似，但边界更低：guest OS 本身认为自己在控制机器。

成功的机器虚拟化至少同时满足三项要求：

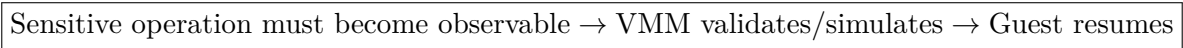
1. **Equivalence**：Guest 观察到的行为与目标机器语义相容；
2. **Resource Control**：VMM 仍掌握真实资源，Guest 不能绕过边界；
3. **Efficiency**：绝大多数普通指令可以直接在硬件执行，不被逐条软件解释。

8.3 CPU 虚拟化：怎样让 Guest 的敏感操作可被 VMM 观察

四条典型路径都在修复同一个可观察性条件：

方案	机制	主要代价/边界
Trap-and-Emulate	Guest 的敏感特权操作触发 trap, VMM 在虚拟状态上模拟效果	只对会 trap 的操作有效
Binary Translation	执行前扫描并改写不会自然 trap 的敏感指令	Guest 无需修改；翻译、缓存和自修改代码处理复杂
Paravirtualization	修改 Guest, 使敏感操作显式发出 hypercall	Guest 必须配合并采用相应接口
Hardware-assisted Virtualization	root/non-root 或同类硬件状态使配置的 Guest 操作产生 VM exit	VM exit/entry 仍有成本，但可观察性由硬件保证

这四种方案不是背诵并列项，而是一条故障修补链：



8.4 内存虚拟化：Guest 物理地址仍不是机器物理地址

机器虚拟化增加了一层地址意义：

GVA → GPA → HPA

Guest OS 负责 GVA → GPA 的虚拟机内地址含义；VMM 与硬件负责 GPA → HPA 的隔离和实际物理地址选择。

影子页表把两级映射在软件中合成为可供硬件消费的 GVA → HPA 视图，主要维护压力落在 Guest 修改页表时；nested paging（如 EPT/NPT）让硬件消费两级映射，降低同步压力，但 TLB miss 时可能需要更多级的 page walk。稳定的 tradeoff 不是“谁绝对更快”，而是把成本放在映射更新路径还是翻译 miss 路径。

8.5 I/O 虚拟化：兼容、批量与直通的陷入频率权衡

方案	数据/控制接口	主要边界
Device Emulation	VMM 模拟具体设备寄存器与行为, Guest 使用原驱动	兼容性高，频繁 trap/emulation 成本高
Paravirtual I/O	Guest 专用驱动通过共享队列批量交接请求	需要 Guest 驱动配合，以批处理减少边界次数
Device Passthrough / SR-IOV	Guest 更直接控制一个物理/虚拟功能	接近原生，但隔离、迁移和资源共享更困难

直通设备可以发起 DMA，因此必须有设备侧地址翻译与保护（典型为 IOMMU），保证设备看到的 DMA address 只能落到被授权的 host physical pages。否则 CPU 页表隔离成立，设备仍可能越界写坏其他 VM。

8.6 Type 1 / Type 2 只作为部署结构理解

经典教材常区分：

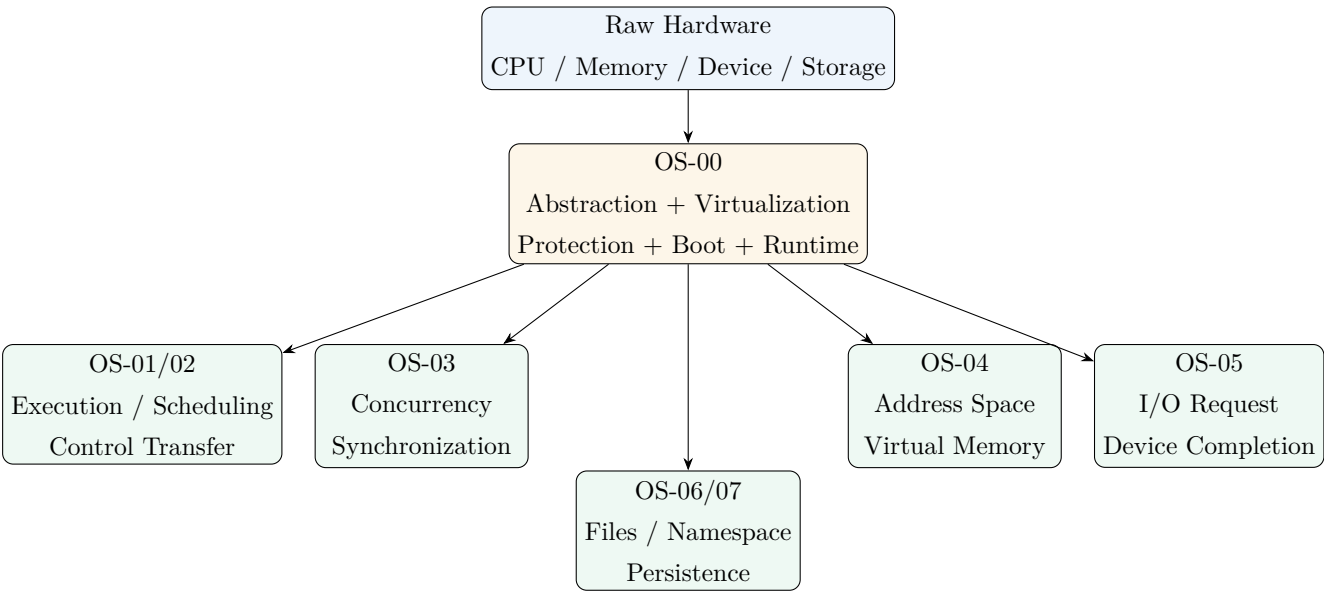
- Type 1: hypervisor 更直接运行在硬件之上；
- Type 2: VMM 作为 host OS 上的软件运行。

不要把分类当成绝对实现定律；现代虚拟化系统可能混合 host kernel module、user-space VMM、hardware virtualization support 等多层组件。408 题按教材抽象判断即可。

最稳定的判据不是产品名称，而是：VMM 使用硬件资源时是否还必须经过另一个通用 host OS 的资源管理路径。KVM 一类混合结构说明两类是架构坐标而不是互斥盒子；考试若给出明确教材抽象，则按题设层次判断。

最后必须阻断一个常见类比：VM 与 process 都提供隔离，但 VM 边界位于 Guest OS 之下、每个 VM 可以运行完整 OS；process 共享同一内核。于是 VM 启动/资源成本通常更高，却能隔离 Guest kernel failure；进程更轻，但共享内核失效会影响所有进程。

9 第九章：一张总图把 OS-00 挂到后续所有 Topic



OSTEP 三条主线也可挂回这个总图：

Virtualization ⇒ OS-01/02 + OS-04 + 部分 OS-05/06

Concurrency ⇒ OS-01/02 + OS-03

Persistence ⇒ OS-05 + OS-06/07

10 第十章：最小调用接口——先定位层，再调用 Owner

OS-00 只负责把问题送进正确的对象层和后续 Owner。第一观察点是：当前谈的是物理资源、受保护机制、OS 抽象、程序可见对象，还是某个已经进入细节的局部子系统？一旦题目开始要求进程状态、同步、页表、I/O 或文件系统内部推演，就停止在本册展开。

完整的概念辨析、七问定位与 Owner 路由统一进入同目录训练文档 基础概念与 Owner 定位.md；Canonical 继续保留 abstraction、virtualization、protection、程序运行环境与启动/结构机制本身。

10.1 忘掉名词时怎样重新推出来？

OS-00 最重要的不是背十几个定义，而是保留一条生成链：

Hardware is finite / complex / shared ⇒ Need abstraction and management

Programs must coexist ⇒ Need virtualization + protection + policy

Activities interleave ⇒ Concurrency + asynchrony problems

Programs start from static files ⇒ Link + Load + Runtime Image

OS itself is software ⇒ Need bootstrapping

Kernel is trusted ⇒ Need a privilege boundary and an architecture choice

Whole OS may itself be virtualized ⇒ Need VMM / Hypervisor

本册最终心智模型

看到“操作系统”时，不再只想到“管理软硬件资源”。应该自动看到四层：

Physical Resource → Protected OS Mechanism
→ Managed Abstraction → Program-visible Environment

然后继续追问：

谁共享？谁隔离？谁决策？什么时候绑定？控制权现在在哪一层？

只要题目进入某个局部机制，就切换到对应 Topic；OS-00 的职责是**把问题送到正确的 Owner**，而不是把所有 OS 知识都重讲一遍。

参考与工程边界来源

- Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Version 1.10: Introduction、Virtualization、Concurrency、Persistence、Virtual Machines。
- System V Application Binary Interface, ELF: *Program Loading and Dynamic Linking*。用于 executable/shared object、process image 与 dynamic linker 的工程边界。
- GNU Binutils, ld documentation: 链接器的 symbol resolution / relocation 基础语义。
- UEFI Specification 2.10, Chapter 3 *Boot Manager*: firmware boot manager 与 OS loader 的启动边界。
- Linux Kernel Documentation, x86 boot protocol: bootloader 与 kernel entry 的工程实例。
- seL4 Documentation / FAQ: microkernel minimal privileged core、user-level services 与 capability-based protection 的工程边界。
- MIT Exokernel research: separating protection from management、application-level resource management 的经典思想。
- 2026 计算机学科专业基础综合（408）考试大纲：操作系统基础、程序运行环境、OS 结构、引导与虚拟机范围。